Representation Learning for Rankings

From Bag-of-Words to Contextualized Representations

Avishek Anand

Information Retrieval

March 2, 2026

Outline

1. From Sparse to Dense — Why We Need Embeddings
2. The Transformer Architecture
3. BERT for Information Retrieval
4. The Two-Stage Retrieve-then-Rerank Pipeline
5. Training Cross-Encoders in Practice
6. Connections and Summary

Section 1

From Sparse to Dense — Why We Need Embeddings

Recap: Where We Are

The Bag-of-Words Legacy enables web-scale retrieval but lacks semantic depth.

- **Efficiency**: Inverted indexes enable sub-second search.
- **Scoring**: BM25 provides a robust lexical baseline.
- **Expansion**: RM3 bridges some lexical gaps via feedback.

The Persistent Problem

Methods rely on **exact term matches**:

- No semantics: *car problems* $\neq$ *automobile issues*.
- Polysemy: *bank* (financial) $=$ *bank* (river).

The Representation Gap

Lexical matching requires exact term overlap to estimate relevance.

- **Sparse**: Each dimension corresponds to a vocabulary word.
- **Dense**: Dimensions represent latent semantic features.
- **Goal**: Learn embeddings where closeness $\approx$ same meaning.

Sparse Representation
$[0, 0, \ldots, 3, \ldots, 0, 2, \ldots, 0]$
$\in \mathbb{R}^{|V|} \quad (|V| \approx 30k)$
$>$99.9% zeros

This lecture $\longrightarrow$

Dense Representation
$[0.21, -0.05, 0.83, \ldots]$
$\in \mathbb{R}^{d} \quad (d = 768)$
100% non-zero

Each dimension = one word
Lexical matching only

Each dimension = latent feature
Semantic matching

The Distributional Hypothesis

Core Principle (Firth, 1957)

"You shall know a word by the company it keeps."

Example: What is a "*tezguino*"?

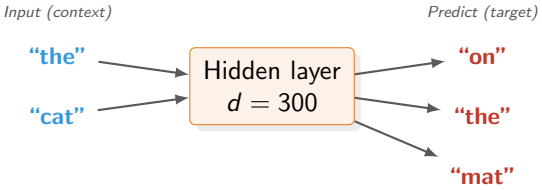
- "A bottle of *tezguino* is on the table."
- "Everybody enjoys *tezguino*."
- "We brew *tezguino* from corn."

Inference: It's an alcoholic drink. Context reveals meaning!

Word2Vec: Words as Vectors

The Skip-gram model learns word meaning from local context windows.

- **Hypothesis**: Words in similar contexts have similar meanings.
- **Task**: Predict surrounding context tokens from a target word.
- **Objective**: Weights in the hidden layer become the embeddings.



After training: *Weights reveal semantic word vectors.*
Words in similar contexts → similar vectors.

Word2Vec: Semantic Properties

Semantic Properties

$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

Why this matters for IR:

- Vectors capture **relational structure**.
- Semantic similarity $\approx$ vector proximity (cosine similarity).

Word2Vec: Relevance to IR

Vector proximity enables matching beyond exact keywords.

- **High Sim**: $\cos(\vec{\text{car}}, \vec{\text{automobile}}) \approx 0.7$.
- **Low Sim**: $\cos(\vec{\text{car}}, \vec{\text{banana}}) \approx 0.1$.
- **Recall**: Enables finding relevant docs with zero term overlap.

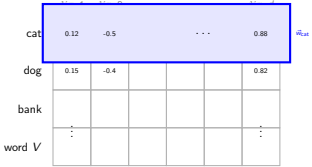
The Naive Approach: $\vec{d} = \frac{1}{|d|} \sum_{w \in d} \vec{w}$

- **Mechanism:** "the cat sat on the mat" $\rightarrow \text{Avg}(\vec{\text{the}}, \vec{\text{cat}}, \ldots)$
- **Pros:** Simple, fast, significantly better than BoW for vocabulary mismatch.

Query Processing with Word Embeddings

How can we build an IR system using word embeddings?

- **Approach**: Represent both the query q and the document d as vectors in the same d-dimensional space.
- **Mechanism**: Use a pre-trained **embedding table** to look up vectors for each word.



Usage in IR:

- **Represent**: $\vec{q}$ and $\vec{d}$ are derived from word vectors (e.g., via averaging).
- **Rank**: Documents are ranked by vector similarity: $\text{score}(q, d) = \cos(\vec{q}, \vec{d})$.

So what is the problem?

The Static Vector Bottleneck

Each word has **exactly one** vector regardless of its context.

- *bank* (river) = *bank* (financial).

Critical Failures

- Loses word order ("dog bites man" = "man bites dog").
- Polysemy unresolved.

Static vs. Contextualized Embeddings

The 2018 evolution moved IR from fixed word vectors to dynamic, context-aware representations.

- **Static**: Word2Vec/GloVe assignment is fixed (ignores neighbors).
- **Contextualized**: BERT embeddings adapt to local word usage.
- **Polysemy**: Distinguishes between "**bank**" (finance) and "**bank**" (river).

Static Embeddings
Word2Vec, GloVe
"bank" $\rightarrow$ **one vector** always
Context-independent

2018 Evolution $\longrightarrow$

Contextualized Embeddings
BERT, RoBERTa, GPT
"bank" $\rightarrow$ **dynamic vector** depending on neighbors
Context-dependent

Section 2

The Transformer Architecture

Why Transformers?

Attention mechanisms enable models to process entire sequences in parallel with global context.

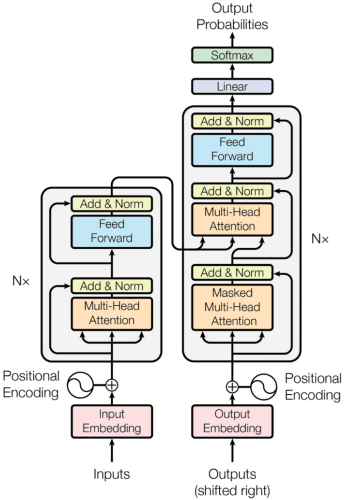
They came after Recurrent Neural Networks (RNNs) and LSTMs that solved the sequence problem but were limited

- **Parallelism**: Processes all tokens simultaneously (unlike sequential RNNs).
- **Recency**: Overcomes "forgetting" in long documents.
- **Scale**: Efficiently trained on massive web-scale corpora.

The SOTA Shift

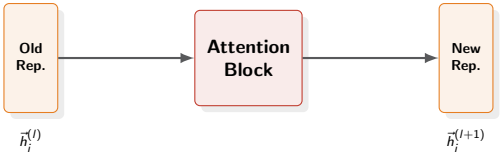
Transformers provide the foundation for every modern high-accuracy retrieval and re-ranking system today.

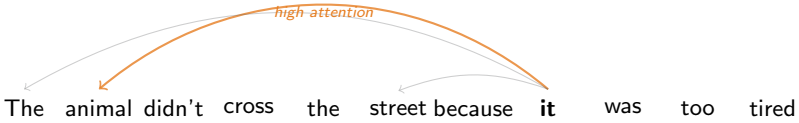
Transformer Architecture



Self-Attention: The Core Idea

Token representations are updated by "attending" to the most relevant neighbors.





Self-Attention: The Mechanism

Interaction is computed via dot-products in Query, Key, and Value spaces.

- **Query** ($\vec{q}$): "What am I looking for in other tokens?"
- **Key** ($\vec{k}$): "What information do I offer to others?"
- **Value** ($\vec{v}$): The actual information content to be aggregated.

The Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Breaking Down the Formula

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Dot Product** (QK^T): Measures raw similarity between tokens.
- **Scaling** ($\sqrt{d_k}$): Stabilizes gradients for deep training.
- **Processing**: Softmax converts scores to weighting factors (sum to 1).

Worked Example: Information Flow

Sentence: "`retrieval models`" ($d_k = 2$)

1. **Similarity**: $\vec{q}_{\text{retr}} \cdot \vec{k}_{\text{retr}} = 0.9$ (Self) $\quad \vec{q}_{\text{retr}} \cdot \vec{k}_{\text{mod}} = 0.6$ (Neighbor)
2. **Weights**: $\text{softmax}([0.9, 0.6]) \approx [\ 0.57\ ,\ \ 0.43\ ]$
3. **Output**: $\vec{h}_{\text{retr}} = 0.57\vec{v}_{\text{retr}} + 0.43\vec{v}_{\text{mod}}$

Interpretation

The representation of "retrieval" is now updated with 43% of the information from "models". Context is integrated!

Multi-Head Attention

Parallel attention heads learn different linguistic and semantic relationships.

- **Heads**: BERT uses 8 heads per layer in parallel.
- **Variety**: Heads learn distinct features (Syntax, Entities, etc.).
- **Robustness**: Aggregating heads provides a richer representation.

Feature Extraction

Calculates 8 different embeddings per token, then concatenates them.

Positional Encoding

Since attention is permutation-invariant, we must inject word order manually.

- **Problem**: Without positions, *dog bites man* = *man bites dog*.
- **Solution**: Add a unique position vector $\vec{p}_i$ to each token $\vec{e}_i$.
- **Result**: Model learns both identity and relative location.

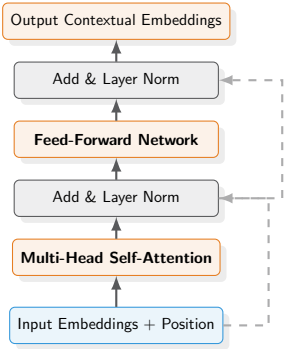
Combined Input Representation

$$\vec{x}_i = \vec{e}_{\text{word}_i} + \vec{p}_i$$

The Transformer Encoder Block

Each encoder block combines parallel attention with per-token feed-forward processing.

- **Attention**: Token interaction via Q, K, V.
- **Residuals**: Prevents gradient signal decay.
- **Norm**: Stabilizes flow through deep layers.



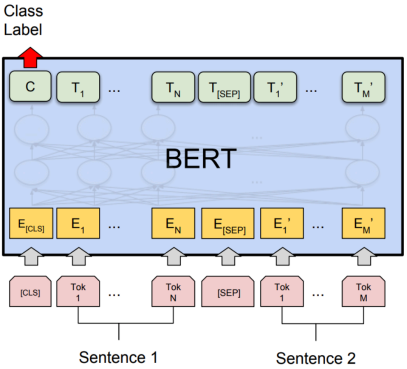
Section 3

BERT for Information Retrieval

What is BERT?

BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only model.

- **Encoder vs. Generative**:
  - **Generative (e.g., GPT)**: Causal; predicts the next token.
  - **Encoder (BERT)**: Bidirectional; sees the entire sequence simultaneously.
- **Context Aware**: Embeddings are computed based on **all** surrounding tokens.
- **Instance**: BERT is a specific implementation of a Transformer Encoder.



Encoder Representation

BERT Architecture: Core Components

The BERT architecture consists of three fundamental layers of processing.

1. **Embeddings Layer**:
   - **BPE / WordPiece**: Splits tokens into sub-words (e.g., *embedding* $\rightarrow$ *embed*, *##ding*).
   - **Positional Encoding**: Injects sequence order info.
2. **Multi-Head Attention (MHA)**:
   - Computes token interactions via Q, K, V dot-products.
   - Every token "attends" to every other token in the sequence.
3. **Stacking & Residuals**:
   - Stacks many identical encoder blocks (12 for Base, 24 for Large).
   - **Residual Stream**: Skip-connections allow signal to flow across layers.

Step 1: Subword Tokenization

BERT processes text by breaking words into frequent sub-words (WordPiece).

- **Query**: "how to rank" → `[how, to, rank]`
- **Doc**: "ranking models" → `[rank, ##ing, models]`
- **Sequence**: `[CLS] Q [SEP] D [SEP]`

The Input Sequence

[*CLS*] how to rank [*SEP*] rank ##ing models [*SEP*]

Step 2: Combining Embeddings

The model must know where tokens are and which sequence they belong to.

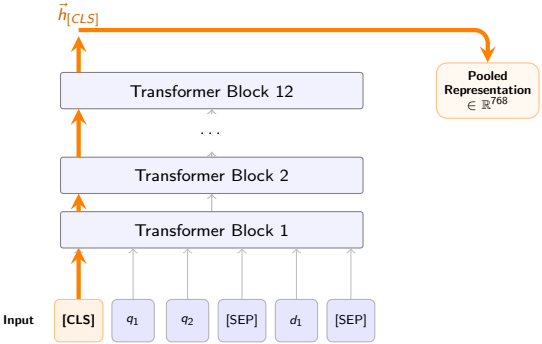
Tokens	[CLS]	the	cat	[SEP]	sat	[SEP]
	$+$	$+$	$+$	$+$	$+$	$+$
Position	E_0	E_1	E_2	E_3	E_4	E_5
	$+$	$+$	$+$	$+$	$+$	$+$
Segment	S_A	S_A	S_A	S_A	S_B	S_B
	$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$	$\Downarrow$
Sum	Input 0	Input 1	Input 2	Input 3	Input 4	Input 5

- **Token**: The sub-word ID from the vocabulary.
- **Position**: A unique vector for index $0, 1, 2, \ldots$
- **Segment**: S_A for Query, S_B for Document.

Step 3: Transformer Stacking to [CLS]

The [CLS] token aggregates context from the entire (q, d) pair.

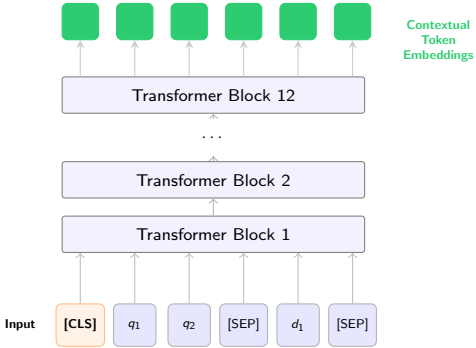
- **Transformation**: Each layer updates all token representations.
- **Context**: Through MHA, the `[CLS]` vector "sees" the relationship between query and doc.
- **Pooling**: The final vector at index 0 becomes our representation.



Alternative: Token-wise Representations

BERT also provides a contextualized vector for every single input token.

- **Rich Context**: Each $\vec{h}_i$ contains the meaning of token i *in that specific sequence*.
- **Applications**:
  - **NER/POS**: Classify each token.
  - **Question Answering**: Predict start/end spans.
  - **Late Interaction**: Used in models like ColBERT.



Pre-training at Scale

BERT was the first model to leverage massive unlabeled data for general language understanding.

- **The Scale Shift**: Trained on Google Books (800M words) + Wikipedia (2.5B words).
- **Masked LM (MLM)**: Hides 15% of tokens to predict from context.
- **Next Sentence Prediction (NSP)**: Learns sequence coherence.

The Result

A "foundation model" that already knows how language works and stores significant **world knowledge**.

Transfer Learning: Fine-tuning as "Orienting"

Transfer learning allows us to adapt a foundation model to specific IR tasks.

- **Pre-training**: Learns general semantics (Expensive, done once).
- **Fine-tuning**: The process of **"orienting"** pre-trained knowledge to a specific task (e.g., Ranking).
- **Efficiency**: Requires much less data and compute than training from scratch.

Analogy

A student graduating from university (Pre-training) is "fine-tuned" when they start a specific job.

What BERT Knows: Layer by Layer

Experiments show that BERT encodes linguistic features hierarchically.

- **Lower Layers**: Surface-level NLP (POS tagging, morphology, syntax).
- **Middle Layers**: Complex semantics and coreference resolution.
- **Upper Layers**: Abstract world knowledge and task-specific logic.

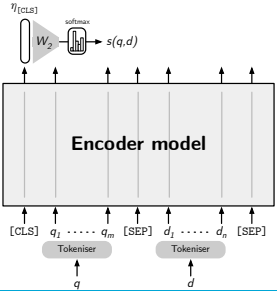
For IR Ranking

Upper layers provide the "deep" semantic matching required to bridge the lexical gap.

Cross-Encoder or MonoBERT: BERT for Re-ranking

Cross-Encoders enable deep token-level interaction between query and document.

- **Input**: Feed query q and document d together as one sequence.
- **Interaction**: Every query token attends to every document token.
- **Scoring**: The final [CLS] token output serves as the relevance score.



Worked Example: Cross-Encoder (1/2)

The model processes query and candidate documents as a single concatenated input sequence.

- **Query**: "side effects of aspirin"
- **Doc A**: Mentions "stomach bleeding" and "adverse reactions".
- **Doc B**: Mentions "stock price" and "sales report".

Sequence Tokenization

- [*CLS*] q [*SEP*] doc A [*SEP*]
- [*CLS*] q [*SEP*] doc B [*SEP*]

Worked Example: Cross-Encoder (2/2)

The Transformer models deep interactions to produce a final relevance probability.

- **High Sim**: Doc A matches the intent (medical effects) via soft-matching.
- **Low Sim**: Doc B repeats the keyword "aspirin" but intent (finance) is wrong.
- **Output**: $P(\text{rel})$ is calculated from the [CLS] embedding.

Interpretation

Doc A: $P(\text{rel}) = \mathbf{0.94}$ vs. **Doc B**: $P(\text{rel}) = \mathbf{0.12}$.
The cross-encoder correctly identifies the semantic match.

Why Cross-Encoders are Accurate

Cross-Encoders enable all-to-all interaction between query and document tokens.

- **Full Interaction**: Every query token attends to every document token.
- **Soft-Matching**: Learned at all 12 layers of the Transformer.
- **Recall**: Finds relevant docs that share zero terms with the query.

Efficiency Trade-off

Maximum accuracy comes at the cost of high latency (500ms scale).

The Latency Bottleneck

Full token interaction makes Cross-Encoders slow for large-scale retrieval.

- **Constraint**: We cannot pre-compute $P(\text{rel}|q, d)$ offline.
- **Inference**: Requires running BERT for every query–doc pair at runtime.
- **Scale**: Feasible only for 100–200 candidates per query.

Candidates	Total Time	Feasible?
1,000,000 (Full corpus)	80 hours	× No
100 (Top-rank)	**500 ms Scale**	✓ **Yes**

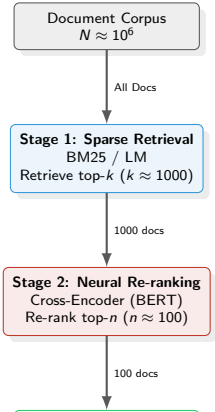
Section 4

The Two-Stage Retrieve-then-Rerank Pipeline

The Two-Stage Pipeline

Efficient neural search requires a multi-stage approach to balance cost and accuracy.

- **Recall**: Sparse methods (BM25) find $\sim$1000 candidates quickly.
- **Precision**: Dense methods (Cross-Encoders) re-rank the top candidates.
- **Scaling**: Enables web-scale search at sub-second latencies.



Why Two Stages?

The pipeline separates the "wide net" retrieval from the "deep focus" ranking.

- **Sparse (Recall)**: Microseconds per query. Metric: Recall@1000.
- **Neural (Precision)**: 5ms scale per doc. Metric: NDCG@10.
- **Combined**: High precision search over an infinite corpus.

The Bounded Recall Problem

If the sparse stage misses a document, the BERT stage can **never** score it. The first stage determines your performance upper-bound.

Comparing Pipeline Variants

Pipeline	NDCG@10	Latency
BM25 Only	0.45	5 ms
BM25 + monoBERT	0.55	500 ms
BM25 + RM3 + monoBERT	**0.57**	520 ms
BM25 + monoT5-3B	**0.58**	2 s

Insight: Neural re-ranking provides the single largest gain in modern IR (10 NDCG points).

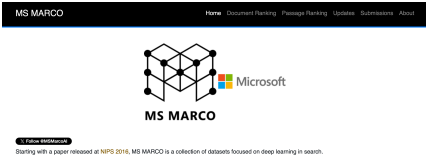
Section 5

Training Cross-Encoders in Practice

The training data: MS MARCO

Neural IR models are typically fine-tuned on the massive MS MARCO benchmark.

- **Scale**: 8.8M web passages and 530k search queries.
- **Labels**: Sparse; usually only one relevant passage per query.
- **Dataset**: Collected from real Bing search logs and human editors.



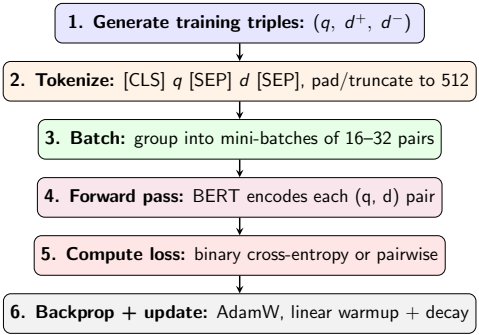
The Neural Catalyst

This benchmark enabled the transition from lexical scoring to deep semantic ranking.

Training Pipeline: Step by Step

The training process iteratively refines the model's ability to rank relevant passages higher.

- **Triples**: Train on (q, d^+, d^-) tuples to learn relative relevance.
- **Loss**: Optimize Contrastive or Cross-Entropy loss functions.
- **Setup**: Typical training takes 12–24 hours on one consumer GPU.



Fine-Tuning BERT for Ranking

Supervised fine-tuning adapts the pre-trained model to specific ranking data.

- **Pointwise**: Classify each pair (query, doc) as relevant or not.
- **Pairwise**: Train so $\text{score}(q, d^+) > \text{score}(q, d^-)$.
- **Data**: Usually requires large benchmarks like MS MARCO.

Common Benchmarks

- **MS MARCO**: 530k queries with sparse (1 passage) labels.
- **TREC**: Smaller collections with dense, multi-level labels.

MS MARCO: Sparse Labels Challenge

Since labels are sparse, models must navigate a sea of unlabeled documents.

- **False Negatives**: Many relevant documents are not labeled.
- **Bias**: We cannot treat all unlabeled docs as truly non-relevant.
- **Sampling**: How we choose the non-relevant pairs defines model quality.

Implication

Training on random negatives is insufficient. Models need challenging, "hard" negatives.

Practice: Training BERT-base

Fine-tuning requires careful initialization and small learning rates.

- **Init**: Start from pre-trained weights (e.g., `bert-base-uncased`).
- **Length**: Must truncate query + doc to 512 sub-word tokens.
- **Schedule**: Learning rate $\approx 2 \times 10^{-5}$ for 24 hours on one GPU.

Production Tip

We are **refining** existing knowledge, not training from scratch!

Standard Models: monoBERT & monoT5

Two dominant architectures define the standard for neural re-ranking pipelines.

- **monoBERT**: Encoder-only; uses the [CLS] token for classification.
- **monoT5**: Encoder-Decoder; uses generative prompts ("Is document relevant?").
- **Choice**: monoT5 is often favored for its flexibility and accuracy.

The SOTA Baseline

Both models consistently outperform lexical methods by a large margin on TREC and MS MARCO.

Comparison: monoBERT vs. monoT5

While both are effective, they differ in architecture and training objectives.

- **monoBERT**: Faster inference; trained via classification.
- **monoT5**: More flexible; trained via sequence-to-sequence generation.
- **Trade-off**: T5's generative approach often yields slightly higher accuracy.

Feature	**monoBERT**	**monoT5**
Architecture	Encoder-only	Encoder-Decoder
Objective	Classification	Text Generation
Inference	Slightly faster	Slightly slower

Common Training Pitfalls

- **Overfitting:** Neural rankers overfit MS MARCO quickly. Stop at 2–3 epochs.
- **Catastrophic Forgetting:** Use small learning rates ($< 3 \times 10^{-5}$) and warmup.
- **Weak Negatives:** Random samples are too easy. Use BM25 hard negatives.

Implementation (Hugging Face)

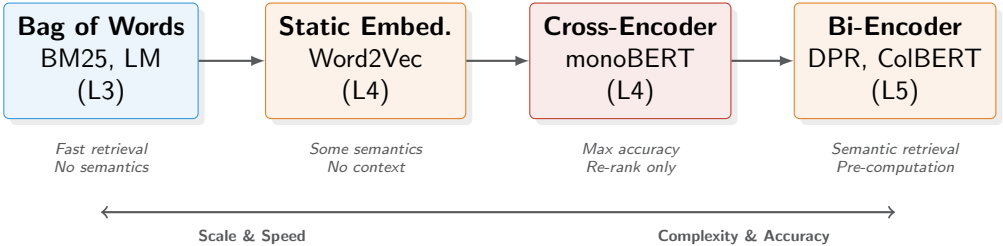
```
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Training args
args = TrainingArguments(
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    num_train_epochs=3,
    fp16=True
)
# Trainer handles batching, eval, and optimization
trainer = Trainer(model=model, args=args, train_dataset=ds)
trainer.train()
```

Section 6

Connections and Summary

The Representation Spectrum

Text representations evolved from sparse matching to deep semantic interaction.

- **Sparse**: High scalability, zero semantics (Exact-match bottleneck).
- **Dense**: Good semantics, high scalability via ANN (Retrieval).
- **Neural**: Deepest interaction, lowest scalability (Re-ranking).